

Java vs Python

A Java developer's quick reference | Mathieu Neron | 2026

This cheatsheet anchors Python on constructs you already know in Java. Left column: Java. Right column: Python (3.10+). Code is illustrative, not always the most idiomatic possible.

1. Hello world & program entry

Program entry point

Java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, " + args[0]);
    }
}
// javac Hello.java && java Hello world
```

Python

```
import sys

def main():
    print(f"Hello, {sys.argv[1]}")

if __name__ == "__main__":
    main()
# python hello.py world
```

File / module layout

Java

```
// One public class per file.
// Filename must match class name.
// src/main/java/com/acme/Hello.java
package com.acme;
public class Hello { ... }
```

Python

```
# Any number of top-level defs/classes per file.
# A file IS a module. A directory with
# __init__.py (or PEP 420 implicit) IS a package.
# acme/hello.py --> import acme.hello
```

2. Variables, constants, type inference

Local variables

Java

```
int x = 5; // primitive
Integer y = 5; // boxed
String s = "hi";
var list = new ArrayList<Integer>(); // Java 10+
final double PI = 3.14159; // immutable binding
```

Python

```
x = 5 # dynamic
x: int = 5 # PEP 526 type hint
s: str = "hi"
nums: list[int] = [] # PEP 585 (3.9+)
PI: Final[float] = 3.14159 # PEP 591
# Convention: ALL_CAPS = "constant" (not enforced)
```

Scope rules

Java

```
// Block-scoped. Shadowing forbidden in same block.
{ int x = 1; }
// 'this.x' for instance, 'ClassName.x' for static.
```

Python

```
# Function-scoped (LEGB: local, enclosing, global,
# built-in). No block scope inside if/for/while.
def f():
    global g_count # rebind module-level
    nonlocal n # rebind enclosing fn
```

3. Primitive types & conversions

Numeric types

Java

```
byte b; short s; int i; long l;    // 8/16/32/64-bit
float f; double d;                // 32/64-bit IEEE
int n = Integer.parseInt("42");
double x = Double.parseDouble("1.5");
String t = String.valueOf(42);
// Overflow wraps silently (use Math.addExact for exc.)
```

Python

```
# int -> arbitrary precision (no overflow)
# float -> 64-bit IEEE
# complex -> a+bj, bool subclasses int
n = int("42")
x = float("1.5")
t = str(42)
big = 2 ** 200                # works fine
```

Bool, char, byte

Java

```
boolean b = true;
char c = 'A';                // 16-bit UTF-16 unit
byte[] bytes = "hi".getBytes(StandardCharsets.UTF_8);
```

Python

```
b = True                    # True / False
c = "A"                    # no char type
bs: bytes = "hi".encode("utf-8") # bytes literal: b"hi"
ba: bytearray = bytearray(b"hi") # mutable
```

4. Strings

Interpolation & formatting

Java

```
String name = "world";
String s1 = "Hello, " + name;
String s2 = String.format("%s = %.2f", "pi", 3.14159);
// Java 15+: text blocks (multi-line)
String body = """
    Hello,
    World
    """;
```

Python

```
name = "world"
s1 = f"Hello, {name}"        # PEP 498 f-string
s2 = "{ } = {:.2f}".format("pi", 3.14159)
s3 = "%s = %.2f" % ("pi", 3.14159) # legacy
body = """
    Hello,
    World
    """                      # multi-line literal
```

Common operations

Java

```
s.length(); s.toUpperCase(); s.trim();
s.startsWith("H"); s.contains("ll");
String[] parts = s.split(",");
String joined = String.join("-", parts);
// Strings IMMUTABLE; use StringBuilder for builds
```

Python

```
len(s); s.upper(); s.strip()
s.startswith("H"); "ll" in s
parts = s.split(",")
joined = "-".join(parts)
# Strings IMMUTABLE; "".join(list) is the build idiom
```

5. Control flow

If / else / ternary

Java

```
if (x > 0) { ... }
else if (x == 0) { ... }
else { ... }

int sign = x > 0 ? 1 : (x < 0 ? -1 : 0);
```

Python

```
if x > 0:
    ...
elif x == 0:
    ...
else:
    ...

sign = 1 if x > 0 else (-1 if x < 0 else 0)
```

Switch / match

Java

```
// Java 14+ switch expression with pattern matching (Java 21)
String label = switch (shape) {
    case Circle c      -> "circle r=" + c.r();
    case Rectangle(var w, var h) -> "rect " + w + "x" + h;
    case null          -> "none";
    default             -> "unknown";
};
```

Python

```
# 3.10+ structural pattern matching (PEP 634)
match shape:
    case Circle(r=r):
        label = f"circle r={r}"
    case Rectangle(w=w, h=h):
        label = f"rect {w}x{h}"
    case None:
        label = "none"
    case _:
        label = "unknown"
```

6. Loops

Counting / classic for

Java

```
for (int i = 0; i < 10; i++) { ... }
for (int i = 9; i >= 0; i--) { ... }
```

Python

```
for i in range(10):          # 0..9
    ...
for i in range(9, -1, -1):   # 9..0
    ...
```

For-each / collection iteration

Java

```
for (String name : names) { ... }
for (var e : map.entrySet()) {
    System.out.println(e.getKey() + "=" + e.getValue());
}
```

Python

```
for name in names:
    ...
for k, v in d.items():      # tuple unpacking
    print(f"{k}={v}")
for i, x in enumerate(xs):  # index + value
    ...
```

While / do-while

Java

```
while (cond) { ... }
do { ... } while (cond);
break; continue;
outer: for (...) { for (...) { break outer; } }
```

Python

```
while cond:
    ...
# No do-while; idiom:
while True:
    ...
    if not cond: break
break; continue
# No labeled break; refactor or use a flag/return
```

7. Collections & data structures

Quick reference

Java	Behavior	Python
<code>ArrayList<T></code>	dynamic array, O(1) get	<code>list</code>
<code>LinkedList<T></code>	doubly-linked, O(1) ends	<code>collections.deque</code>
<code>int[]</code> / <code>Object[]</code>	fixed-size primitive array	<code>array.array("i", ...)</code> (typed)
<code>HashSet<T></code>	unordered set, O(1) ops	<code>set</code>
<code>LinkedHashSet<T></code>	insertion-ordered set	<code>dict.fromkeys(xs)</code> (3.7+ keeps order)
<code>TreeSet<T></code>	sorted set	<code>sortedcontainers.SortedSet</code> (PyPI)
<code>EnumSet</code>	bit-set over enum	set of <code>Enum</code> members
<code>HashMap<K,V></code>	unordered map	dict (insertion-ordered since 3.7!)
<code>LinkedHashMap<K,V></code>	insertion-ordered map	dict (or <code>collections.OrderedDict</code> for <code>move_to_end</code>)
<code>TreeMap<K,V></code>	sorted map	<code>sortedcontainers.SortedDict</code> (PyPI)
<code>IdentityHashMap</code>	identity-keyed map	<code>{id(k): v}</code> or <code>weakref.WeakKeyDictionary</code>
<code>EnumMap</code>	map keyed by enum	plain dict keyed on <code>Enum</code>
<code>WeakHashMap</code>	weakly-referenced keys	<code>weakref.WeakKeyDictionary</code>
<code>ArrayDeque (queue)</code>	FIFO queue, ring buffer	<code>collections.deque</code>
<code>ArrayDeque (stack)</code>	LIFO stack	list via <code>append/pop</code>
<code>ArrayDeque</code>	double-ended queue	<code>collections.deque</code>
<code>PriorityQueue<T></code>	binary heap (min-heap)	<code>heapq</code> on a list
<code>Stack<T></code>	legacy LIFO (synchronized)	list via <code>append/pop</code>
<code>ConcurrentHashMap</code>	thread-safe map	dict + <code>threading.Lock</code> (or shared via <code>multiprocessing.Manager</code>)
<code>BlockingQueue</code>	thread-safe FIFO + blocking	<code>queue.Queue</code>
<code>LinkedBlockingDeque</code>	blocking deque	<code>queue.Queue</code> (no native deque variant)
<code>DelayQueue</code>	expiry-ordered queue	<code>sched</code> module / hand-roll on <code>heapq</code>
<code>List.copyOf</code> , <code>Map.of</code>	shallow-immutable views	tuple, <code>frozenset</code> , <code>MappingProxyType</code>

Lists

ArrayList / LinkedList / array

```
Java
List<Integer> a = new ArrayList<>();           // O(1) get
a.add(1); a.get(0); a.remove(0); a.size();

List<Integer> ll = new LinkedList<>();         // O(1) ends
ll.addFirst(0); ll.addLast(2);

int[] arr = new int[10];                     // fixed
List<Integer> immut = List.of(1, 2, 3);
```

```
Python
xs = [1, 2, 3]                               # dynamic array
xs.append(4); xs[0]; xs.pop(0); len(xs)
xs[1:3]; xs[::-1]                            # slice / reverse view-copy

from collections import deque
dq = deque([1,2,3])                          # O(1) at both ends
dq.appendleft(0); dq.append(4)

import array
a = array.array("i", [1,2,3])                # typed C-contiguous
t = (1, 2, 3)                               # immutable tuple
```

Sets

HashSet / LinkedHashSet / TreeSet

```
Java
Set<String> hs = new HashSet<>();             // unordered
Set<String> ls = new LinkedHashSet<>();       // ins-order
Set<String> ts = new TreeSet<>();             // sorted
ts.first(); ts.headSet("m"); ts.subSet("a","z");

EnumSet<Day> weekend = EnumSet.of(SAT, SUN);
```

```
Python
s = {"a", "b"}; s.add("c"); s.discard("a")
s | t; s & t; s - t; s ^ t                    # ops
fs = frozenset(s)                            # immutable

# Insertion order: dict keys preserve, set does NOT.
ordered = list(dict.fromkeys(["b","a","b","c"]))

# Sorted set: 3rd-party
from sortedcontainers import SortedSet
ts = SortedSet(["c","a","b"]); ts[0]; ts.irange("a","m")
```

Maps

HashMap / LinkedHashMap / TreeMap

Java

```
Map<String,Integer> hm = new HashMap<>(); // unord
Map<String,Integer> lm = new LinkedHashMap<>(); // ins-ord
Map<String,Integer> tm = new TreeMap<>(); // sorted

hm.put("a", 1); hm.getOrDefault("b", 0);
hm.computeIfAbsent("c", k -> 42);
hm.merge("a", 1, Integer::sum); // tally

tm.firstKey(); tm.headMap("m"); tm.floorKey("g");
```

Python

```
# dict preserves INSERTION ORDER (3.7+). So plain dict
# already covers HashMap AND LinkedHashMap.
d = {"a": 1}
d.get("b", 0) # getOrDefault
d.setdefault("c", 42) # computeIfAbsent

# computeIfAbsent for collection-valued maps:
from collections import defaultdict
groups = defaultdict(list); groups["k"].append("v")

# Tally / merge:
from collections import Counter
c = Counter("aabbcccd"); c.most_common(2)

# Sorted map: 3rd-party
from sortedcontainers import SortedDict
sm = SortedDict({"b":2,"a":1}); sm.irange("a","m")
```

Identity / weak / read-only / enum keys

Java

```
Map<Key,V> id = new IdentityHashMap<>(); // == not equals
Map<Key,V> wk = new WeakHashMap<>(); // GC-able keys
Map<K,V> ro = Collections.unmodifiableMap(m); // view
Map<Day,V> em = new EnumMap<>(Day.class);
```

Python

```
import weakref
wk = weakref.WeakKeyDictionary() # WeakHashMap
wv = weakref.WeakValueDictionary() # values can vanish

from types import MappingProxyType
ro = MappingProxyType({"a": 1}) # read-only view

# IdentityHashMap idiom:
id_map = {}
id_map[id(obj)] = value

# EnumMap analog: just dict[Enum, V]
from enum import Enum
class Day(Enum): MON=1; TUE=2
schedule: dict[Day, str] = {}
```

Queue, deque, stack, priority queue

Queue (FIFO) & Deque

Java

```
Deque<Integer> q = new ArrayDeque<>(); // recommended FIFO
q.offer(1); q.offer(2);
int head = q.poll(); // null if empty
int peek = q.peek();

// Double-ended:
q.offerFirst(0); q.offerLast(3);
q.pollFirst(); q.pollLast();

// Capacity-bounded blocking:
BlockingQueue<Integer> bq = new ArrayBlockingQueue<>(8);
bq.put(42); int v = bq.take(); // blocks
```

Python

```
from collections import deque
dq = deque(maxlen=8) # optional cap (drops old)
dq.append(1); dq.appendleft(0) # O(1) both ends
dq.popleft() # FIFO take
dq.pop() # LIFO take
dq[0] # peek

# Thread-safe blocking queue:
import queue
bq = queue.Queue(maxsize=8)
bq.put(42) # blocks if full
v = bq.get() # blocks if empty
# Variants: queue.LifoQueue, queue.PriorityQueue
```

Stack (LIFO)

Java

```
// Avoid legacy java.util.Stack (synchronized + Vector).
Deque<Integer> st = new ArrayDeque<>();
st.push(1); st.push(2);
int top = st.peek();
int v = st.pop();
```

Python

```
# Idiomatic: just a list with append/pop.
st = []
st.append(1); st.append(2)
top = st[-1]
v = st.pop() # O(1) at the end
# (Use deque if you need both ends; list is faster for LIFO.)
```

PriorityQueue (heap)

Java

```
PriorityQueue<Integer> pq = new PriorityQueue<>(); // min
PriorityQueue<Job> jobs =
    new PriorityQueue<>(Comparator.comparingInt(Job::priority));
pq.offer(3); pq.offer(1); pq.offer(2);
int min = pq.poll(); // 1
```

Python

```
import heapq
h = []
heapq.heappush(h, 3); heapq.heappush(h, 1); heapq.heappush(h, 2)
mn = heapq.heappop(h) # 1 (min-heap)

# Custom priority: push (priority, item) tuples
heapq.heappush(h, (job.priority, job))

# Max-heap: push negated keys, or use a wrapper.
# Thread-safe: queue.PriorityQueue.
```

Concurrent & specialized

Concurrent maps and queues

Java

```
ConcurrentHashMap<String,Long> chm = new ConcurrentHashMap<>();
chm.merge("k", 1L, Long::sum); // atomic tally
chm.computeIfAbsent("k", k -> heavy()); // single-flight

BlockingQueue<Task> q = new LinkedBlockingQueue<>();
q.put(t); Task next = q.take();

CopyOnWriteArrayList<Listener> listeners
    = new CopyOnWriteArrayList<>(); // read-mostly
```

Python

```
# No native lock-free map. Patterns:
import threading
lock = threading.Lock()
counts = {}
with lock:
    counts["k"] = counts.get("k", 0) + 1

# Thread-safe queue:
import queue
q = queue.Queue()
q.put(t); next_ = q.get()

# Cross-process state:
import multiprocessing
m = multiprocessing.Manager().dict()
```

Specialized: Counter, ChainMap, NamedTuple

Java

```
// Tally: Map<K,Long> + merge, or Stream.collect(groupingBy)
Map<String,Long> tally = words.stream()
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()));

// Layered config: walk a list of Maps yourself.
// Named tuple: record (Java 16+)
record Point(int x, int y) {}
```

Python

```
from collections import Counter, ChainMap, namedtuple, OrderedDict
c = Counter(words) # multiset / tally
c["x"] += 1; c.most_common(3)

cfg = ChainMap(env, file_cfg, defaults) # layered lookup

P = namedtuple("Point", "x y") # immutable record
# Modern: dataclasses.dataclass(frozen=True) is preferred.

od = OrderedDict() # adds .move_to_end / popitem(last
    ↪ )
od["a"] = 1; od.move_to_end("a")
```

Mutability defaults

Java: ArrayList, HashMap, HashSet mutable. List.of / Set.of / Map.of return true-immutable instances. Collections.unmodifiableX wraps a *view* — the underlying can still be mutated by holders of the original.

Python: list, dict, set mutable. tuple, frozenset, bytes, str immutable. MappingProxyType wraps a dict for a shallow-readonly view. For "deeply readonly" data: @dataclass(frozen=True), NamedTuple, or tuple-of-tuples.

8. Functions

Definition, default args, varargs

Java

```
int add(int a, int b) { return a + b; }  
// no default args: use overloading  
int add(int a) { return add(a, 1); }  
// varargs:  
int sum(int... xs) {  
    int t = 0; for (int x : xs) t += x; return t;  
}  
sum(1, 2, 3);
```

Python

```
def add(a: int, b: int = 1) -> int:  
    return a + b  
  
def sum_(*xs: int) -> int:  
    return sum(xs)  
  
def cfg(**kw): # kwargs dict  
    print(kw)  
  
add(2) # b defaults to 1  
sum_(1, 2, 3)  
cfg(host="x", port=80)
```

Multiple return values

Java

```
// Records (Java 16+) for ad-hoc tuples.  
record Pair<A,B>(A first, B second) {}  
Pair<Integer,String> divmod(int x, int y) {  
    return new Pair<>(x / y, x + " over " + y);  
}
```

Python

```
def divmod_(x: int, y: int) -> tuple[int, int]:  
    return x // y, x % y  
  
q, r = divmod_(10, 3) # tuple unpacking
```

9. Lambdas, closures, higher-order

Lambda & method reference

Java

```
Function<Integer,Integer> sq = x -> x * x;  
Predicate<String> nonEmpty = s -> !s.isEmpty();  
BiFunction<Integer,Integer,Integer> add = (a,b) -> a+b;  
// Method ref:  
list.forEach(System.out::println);
```

Python

```
sq = lambda x: x * x # one-expression only  
non_empty = lambda s: bool(s)  
add = lambda a, b: a + b  
  
# More common: def + first-class function passing  
for x in xs: print(x) # no need for forEach
```

Streams / functional pipelines

Java

```
List<Integer> result = xs.stream()  
    .filter(x -> x > 0)  
    .map(x -> x * 2)  
    .sorted()  
    .toList();  
int total = xs.stream().mapToInt(Integer::intValue).sum();
```

Python

```
result = sorted(x * 2 for x in xs if x > 0)  
total = sum(xs)  
  
# Or via map/filter (lazy iterators):  
list(map(lambda x: x*2, filter(lambda x: x>0, xs)))  
# Idiomatic Python prefers comprehensions.
```

10. Classes

Class with fields, constructor, methods

Java

```
public class Point {  
    private final double x, y;  
    public Point(double x, double y) {  
        this.x = x; this.y = y;  
    }  
    public double dist(Point o) {  
        double dx = x - o.x, dy = y - o.y;  
        return Math.sqrt(dx*dx + dy*dy);  
    }  
}
```

Python

```
class Point:  
    def __init__(self, x: float, y: float):  
        self.x = x  
        self.y = y  
  
    def dist(self, o: "Point") -> float:  
        dx, dy = self.x - o.x, self.y - o.y  
        return (dx * dx + dy * dy) ** 0.5
```

Records / data classes

Java

```
// Java 16+
public record Point(double x, double y) {
    public double dist(Point o) {
        return Math.hypot(x - o.x, y - o.y);
    }
}
// Auto: equals, hashCode, toString, accessors
```

Python

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float
    def dist(self, o: "Point") -> float:
        return ((self.x-o.x)**2 + (self.y-o.y)**2)**0.5
# Auto: __init__, __repr__, __eq__, __hash__
```

Inheritance, abstract, interfaces

Java

```
abstract class Shape {
    abstract double area();
    public String name() { return "Shape"; }
}
interface Drawable { void draw(); }
class Circle extends Shape implements Drawable {
    private final double r;
    public Circle(double r) { this.r = r; }
    @Override double area() { return Math.PI*r*r; }
    @Override public void draw() { ... }
}
```

Python

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
    def name(self) -> str: return "Shape"

class Drawable(ABC):
    @abstractmethod
    def draw(self) -> None: ...

class Circle(Shape, Drawable): # multiple inherit OK
    def __init__(self, r: float): self.r = r
    def area(self): return 3.14159 * self.r * self.r
    def draw(self): ...
```

Static / class methods

Java

```
public class MathU {
    public static int sq(int x) { return x*x; }
    public static final double PI = 3.14159;
}
MathU.sq(3);
```

Python

```
class MathU:
    PI = 3.14159

    @staticmethod
    def sq(x: int) -> int: return x * x

    @classmethod
    def named(cls, x): return cls.sq(x)

MathU.sq(3)
```

11. Generics / type parameters

Generic class & method

Java

```
class Box<T> {
    private final T value;
    public Box(T v) { this.value = v; }
    public T get() { return value; }
}
<T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}
List<? extends Number> ns; // bounded wildcard
```

Python

```
# 3.12+ syntax (PEP 695)
class Box[T]:
    def __init__(self, v: T): self.value = v
    def get(self) -> T: return self.value

def max_[T: "SupportsRichComparison"](a: T, b: T) -> T:
    return a if a >= b else b

# Pre-3.12: TypeVar from typing
from typing import TypeVar
T = TypeVar("T")
```


12. Error handling

Try / catch / finally

Java

```
try (var f = Files.newBufferedReader(p)) {
    return f.readLine();
} catch (IOException | RuntimeException e) {
    log.error("read failed", e);
    throw new MyException("oops", e);
} finally {
    cleanup();
}
// Checked exceptions force `throws` declarations.
```

Python

```
try:
    with open(p) as f:
        return f.readline()
except (OSError, RuntimeError) as e:
    log.error("read failed", exc_info=e)
    raise MyError("oops") from e
finally:
    cleanup()
# All exceptions unchecked. No `throws` clause.
```

Custom exceptions

Java

```
public class NotFoundException extends RuntimeException {
    public NotFoundException(String msg) { super(msg); }
}
throw new NotFoundException("user " + id);
```

Python

```
class NotFoundError(Exception):
    pass

raise NotFoundError(f"user {id}")
```

13. Modules, packages, imports

Imports

Java

```
package com.acme.svc;
import java.util.List;
import java.util.stream.Collectors;
import static java.util.Map.entry;

// Maven / Gradle: declare in pom.xml / build.gradle
```

Python

```
from typing import List
from collections import defaultdict
import json
import numpy as np

# Pip / Poetry / uv: declare in pyproject.toml
# pip install requests / uv add requests
```

14. Concurrency

Threads

Java

```
Thread t = new Thread(() -> work());
t.start(); t.join();

// Pools:
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> compute());
int v = f.get();
pool.shutdown();
```

Python

```
import threading
t = threading.Thread(target=work); t.start(); t.join()

# Pools (CPU-bound -> processes; I/O -> threads):
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(4) as pool:
    f = pool.submit(compute); v = f.result()
# GIL: pure-Python CPU work doesn't parallelize across
# threads; use multiprocessing or release GIL via C ext.
```

Java

```
CompletableFuture<String> fut =
    CompletableFuture.supplyAsync(() -> fetch())
        .thenApply(String::trim)
        .exceptionally(e -> "fallback");
String r = fut.get();
```

Python

```
import asyncio

async def run():
    try:
        r = (await fetch()).strip()
    except Exception:
        r = "fallback"
    return r

asyncio.run(run()) # entry point
# async def + await; coroutines run on an event loop.
```

15. Idiom appendix

Null / None / equality

Java

```
if (s == null) { ... } // identity
if (a.equals(b)) { ... } // value
Optional<String> os = Optional.ofNullable(s);
String x = os.orElse("default");
```

Python

```
if s is None: ... # identity (always 'is')
if a == b: ... # __eq__
x = s if s is not None else "default"
# Optional[T] in typing == Union[T, None] == T | None (3.10+)
```

Printing & logging

Java

```
System.out.println("hi");
System.err.printf("x=%d\n", x);
Logger log = LoggerFactory.getLogger(C.class);
log.info("user {} logged in", uid);
```

Python

```
print("hi")
print(f"x={x}", file=sys.stderr)
import logging
log = logging.getLogger(__name__)
log.info("user %s logged in", uid)
```

Common gotchas (Python coming from Java)

- **Default arg values are evaluated once.** `def f(x=[])` reuses the same list across calls. Use `None` sentinels: `def f(x=None): x = x or []`.
- **Indentation is syntax.** Mixing tabs and spaces is a real bug. Configure your editor.
- **Slicing copies.** `xs[:]` is a shallow copy; `xs[a:b] = []` mutates in place.
- **No method overloading.** Latest `def` wins. Use default args or `functools.singledispatch`.
- **Truthiness is broad.** `0`, `""`, `[]`, `{}`, `None` are all falsy. Prefer `is None` for null checks.
- **Equality.** `==` calls `__eq__` (like Java's `equals`). `is` is identity (like Java's `==`).
- **No private/protected.** Convention: leading `_` = internal, `__name` = name-mangled.
- **Mutable default state in classes.** `class C: items = []` shares the list across instances (it's class-level). Use `__init__`.